Activity 5: Technology Architecture Design

IT Asset Management Transformation Using Snipe-IT

Activity Overview

Activity Title: Technology Architecture Design for IT Asset Management

Objective: Students will design the technology architecture for their fictional company's IT asset management transformation, building on the TO-BE business processes (Activity 2), data architecture (Activity 3), and application architecture using the C4 model (Activity 4). This activity focuses on mapping containers to physical and cloud infrastructure, defining network topology, selecting technology stacks, and integrating managed services for AI/ML and polyglot persistence.

Duration: 2-3 weeks

Weight: 15% of course grade

Prerequisite Activities: Activity 2 (TO-BE Business Architecture), Activity 3 (Data Architecture), Activity 4 (Application Architecture - C4 Model)

Why Technology Architecture?

While application architecture (C4 Level 2) defines *what* containers exist and *how* they communicate, technology architecture defines *where* they run and *what* specific technologies power them. This is the layer where decisions about cloud providers, infrastructure as code, networking, security, and managed services are made.

The technology architecture answers critical questions:

- **Where does each container run?** (Cloud, on-premises, hybrid)
- **How do containers scale?** (Horizontal, vertical, auto-scaling)
- **How is the network secured?** (Firewalls, subnets, security groups)

- **What specific technologies are selected?** (AWS vs. Azure vs. GCP, Kubernetes vs. ECS, etc.)
- **What managed services reduce operational overhead?** (RDS, SageMaker, MongoDB Atlas, etc.)

For the PUP IT Asset Management transformation, the technology architecture must support the polyglot persistence layer (PostgreSQL, MongoDB, InfluxDB, Neo4j), the AI analytics layer (predictive maintenance, demand forecasting), and the integration layer (Kafka, API gateway).

Summary of Deliverables

Deliverable	Format	Description
Deliverable 1	Diagram + Document	Deployment Architecture Diagram (Containers to Infrastructure)
Deliverable 2	Diagram + Document	Network Topology and Security Architecture
Deliverable 3	Spreadsheet/Document	Technology Stack Selection Matrix
Deliverable 4	Document	Managed Services Selection and Justification
Deliverable 5	Document	Infrastructure as Code Strategy (Terraform/Pulumi)
Deliverable 6	Document	Scalability and High Availability Design
Deliverable 7	Spreadsheet	Cost Estimation and Budget Analysis

Deliverable 8	Spreadsheet	Traceability Matrix (Technology Decisions to Requirements)
Deliverable 9	Presentation	Executive Summary Slides

Deliverable 1: Deployment Architecture Diagram

Format

- Professional diagram (Lucidchart, Draw.io, Structurizr, or similar)
- Accompanying documentation (3-4 pages)
- Filename: `TeamName_Deployment_Architecture.pdf` and `TeamName_Deployment_Architecture_Source`

Description

The Deployment Architecture Diagram maps each container from the C4 Container Diagram (Activity 4) to specific infrastructure resources—whether physical servers, virtual machines, cloud instances, or managed services. This diagram answers: Where does each container run? How are containers grouped? What is the geographic distribution?

Requirements

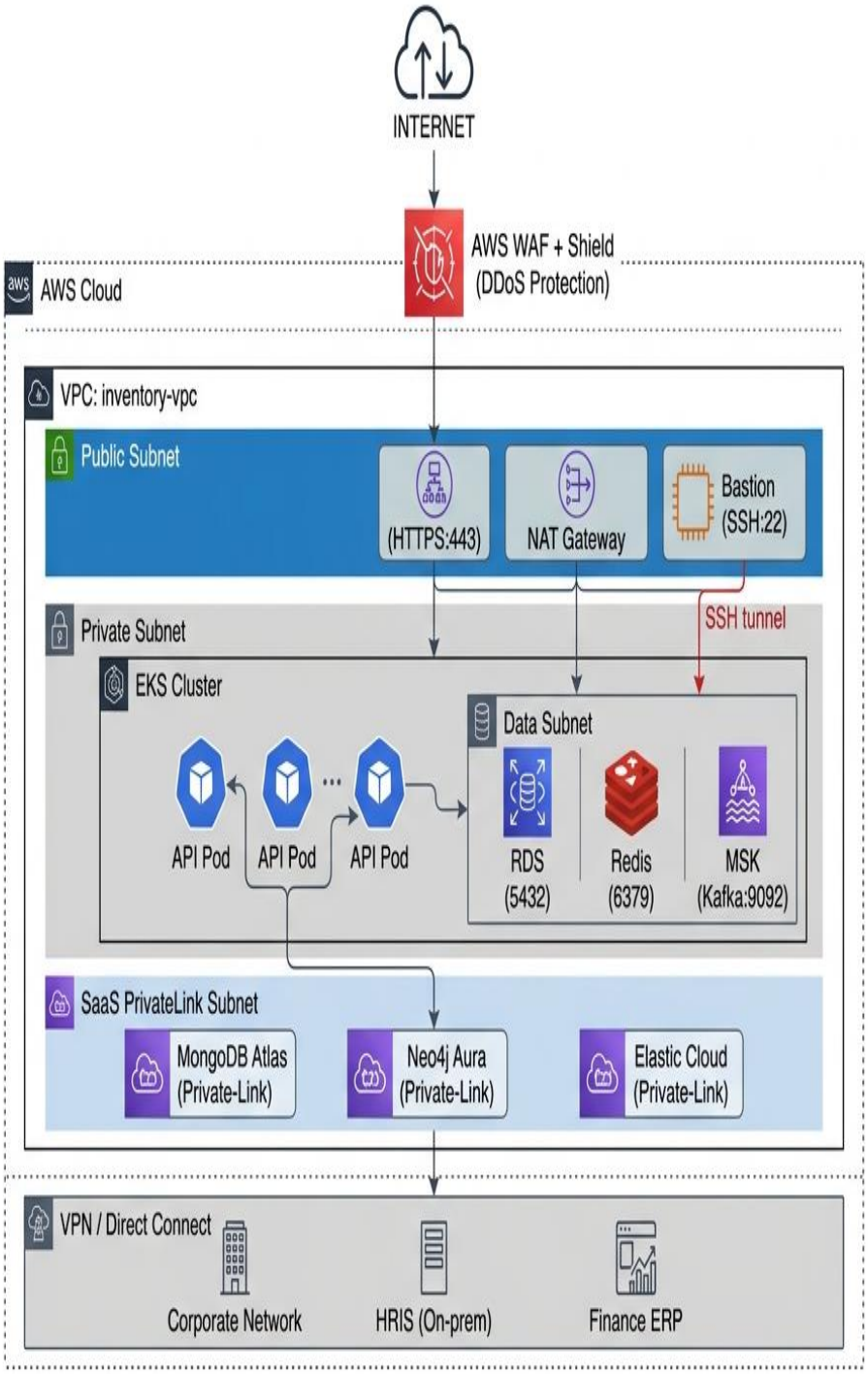
1. Container to Infrastructure Mapping

For each container identified in Activity 4, specify its deployment target:

Container	Deployment Target	Instance Type	Scaling	Availability Zone	Data Residency
Web Application	AWS S3 + CloudFront	Static hosting	Global CDN	Multi-region	Content delivery
Mobile App	App Store / Play Store	N/A	N/A	Global	N/A
API Gateway	AWS API Gateway / Kong	Managed	Auto	Multi-AZ	Regional
API Application	AWS EKS (Kubernetes)	t3.large (orchestration)	Horizontal (3-10 pods)	Multi-AZ	Regional
PostgreSQL (Snipe-IT)	AWS RDS	db.t3.large	Vertical + Read replicas	Multi-AZ	Regional
MongoDB	MongoDB Atlas	M30	Horizontal sharding	Multi-region	Regional
InfluxDB	InfluxDB Cloud	Starter tier	Auto-scaling	Regional	Regional
Neo4j	Neo4j Aura	Professional	Vertical	Regional	Regional
Redis	AWS ElastiCache	cache.t3.micro	Cluster mode	Multi-AZ	Regional

Kafka	AWS MSK / Confluent	kafka.t3.small	Auto-scaling	Multi-AZ	Regional
Elasticsearch	Elastic Cloud	aws.data.highio	Hot-warm architecture	Multi-AZ	Regional
Debezium	Kafka Connect	Same as Kafka	Managed connector	Multi-AZ	Regional
Predictive Service	AWS EKS	t3.medium	Horizontal (2-5 pods)	Multi-AZ	Regional
Telemetry Service	AWS EKS	t3.medium	Horizontal (2-5 pods)	Multi-AZ	Regional
Workflow Engine	Temporal Cloud	Standard	Managed	Multi-region	Regional
Batch Processor	AWS MWAA (Airflow)	mwaa.small	Auto-scaling	Single-AZ	Regional

2. Deployment Architecture Diagram Example



3. Deployment Documentation

For each deployment component, document:

Component	Deployment Details	Configuration	High Availability	Backup/DR
EKS Cluster	Managed Kubernetes	3 nodes min, auto-scaling to 10	Multi-AZ	Cluster state backed up
RDS PostgreSQL	Managed database	Multi-AZ deployment, automated backups	Failover replica in different AZ	Daily snapshots, 30-day retention
MongoDB Atlas	SaaS database	M30 cluster, 3 nodes	Multi-zone	Continuous backups
InfluxDB Cloud	SaaS time-series	Starter tier	Cloud-managed	Cloud-managed
Neo4j Aura	SaaS graph	Professional tier	Cloud-managed	Cloud-managed
ElastiCache Redis	Managed cache	Cluster mode enabled	Multi-AZ with auto-failover	Daily snapshots
MSK Kafka	Managed Kafka	3 broker nodes	Multi-AZ	7-day retention
MWAA Airflow	Managed workflow	mwaa.small	Single-AZ (can be upgraded)	S3-backed DAG storage

Deliverable 2: Network Topology and Security Architecture

Format

- Professional diagram with accompanying documentation (3-4 pages)
- Filename: `TeamName_Network_Security.pdf`

Description

The Network Topology and Security Architecture defines how components communicate, where security boundaries exist, and how data is protected in transit and at rest. This deliverable answers: How are subnets segmented? What firewall rules apply? How is access controlled? What encryption is used?

Requirements

1. Network Segmentation

Define the network layers and their purposes:

Network Layer	CIDR	Purpose	Access
Public Subnet	10.0.1.0/24	Load balancers, NAT gateways, bastion hosts	Internet inbound (restricted)
Private Subnet - Application	10.0.2.0/24	EKS nodes (API, Predictive, Telemetry)	Only from load balancer
Private Subnet - Data	10.0.3.0/24	RDS, ElastiCache, MSK	Only from application subnet
Private Subnet - SaaS	10.0.4.0/24	Managed service endpoints	PrivateLink connections

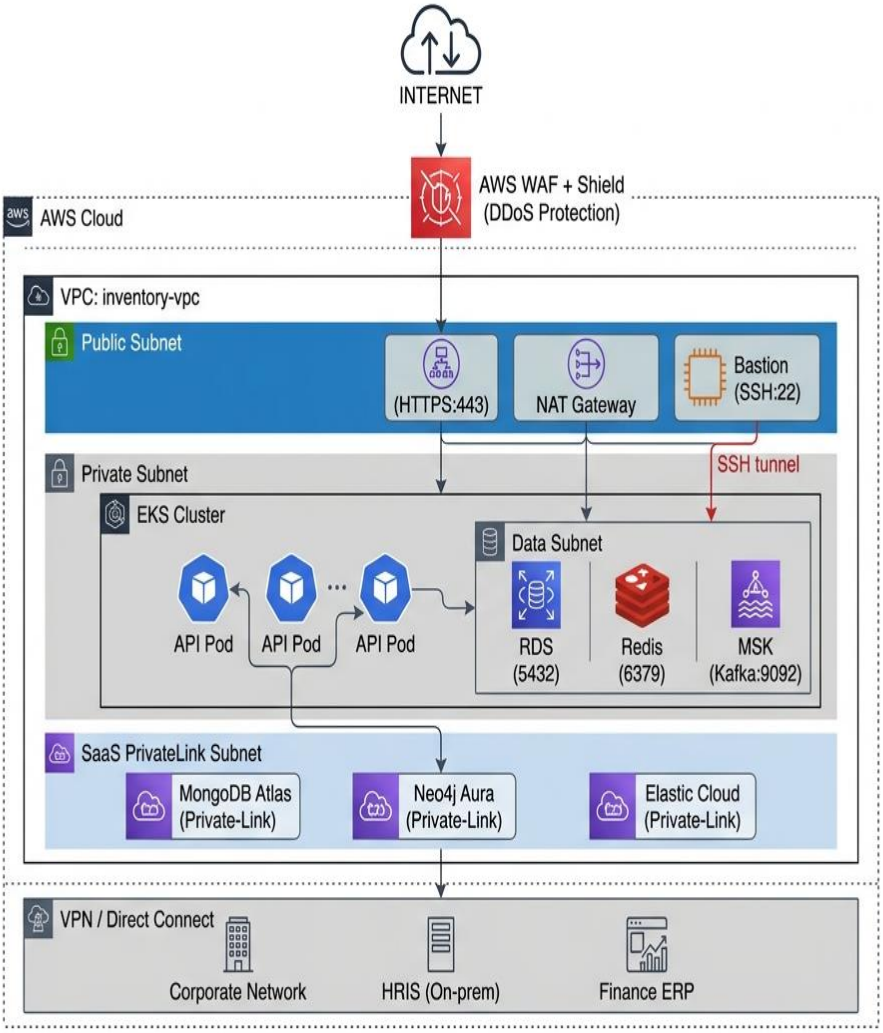
Network Layer	CIDR	Purpose	Access
Private Subnet - Batch	10.0.5.0/24	MWAA Airflow	Only from application subnet

2. Security Group Rules

Define security groups for each component:

Security Group	Inbound Rules	Outbound Rules	Description
Load Balancer SG	HTTPS (443) from 0.0.0.0/0, HTTP (80) for redirect	HTTPS to API SG	Internet-facing traffic
API SG	HTTPS (443) from Load Balancer SG, SSH (22) from Bastion SG	HTTPS to internet (external APIs), DB ports to Data SG	API application instances
Database SG	PostgreSQL (5432) from API SG, Redis (6379) from API SG, Kafka (9092) from API SG	None (locked down)	Data layer components
Bastion SG	SSH (22) from corporate IP range (203.177.x.x)	SSH to API SG, DB SG	Administrative access
SaaS Endpoint SG	PrivateLink connections from VPC	None	MongoDB, Neo4j, Elastic Cloud

3. Network Security Diagram Example



4. Data Encryption Strategy

Data Category	At Rest Encryption	In Transit Encryption	Key Management
RDS PostgreSQL	AWS RDS encryption (AES-256)	TLS 1.2+	AWS KMS

Data Category	At Rest Encryption	In Transit Encryption	Key Management
MongoDB Atlas	Atlas encryption (AES-256)	TLS 1.2+	Cloud provider KMS
InfluxDB Cloud	Platform encryption	TLS 1.2+	Cloud provider
Neo4j Aura	Platform encryption	TLS 1.2+	Cloud provider
S3 Static Assets	S3-SSE (AES-256)	TLS 1.2+	AWS KMS
EBS Volumes	AWS EBS encryption	N/A	AWS KMS
API Traffic	N/A	TLS 1.2+	Certificate Manager
Backups	Encrypted snapshots	TLS 1.2+	AWS KMS

Deliverable 3: Technology Stack Selection Matrix

Format

- Spreadsheet or table in document
- Filename: `TeamName_Technology_Stack.xlsx`

Description

The Technology Stack Selection Matrix documents all technology choices for the infrastructure, with justifications for each selection based on criteria such as cost, performance, team skills, scalability, and integration capabilities.

Requirements

Component	Technology	Version	Justification	Alternatives Considered	Selection Rationale
Cloud Provider	AWS	-	Broadest service offering, mature AI/ML services, strong regional presence	Azure, GCP	AWS has best SageMaker for AI, RDS for PostgreSQL, and MSK for Kafka
Container Orchestration	EKS (Kubernetes)	1.28	Portability, ecosystem, auto-scaling, service discovery	ECS, Fargate	Kubernetes skills transferable; EKS managed service reduces ops
Compute (API)	EKS nodes (t3.large)	-	Balanced CPU/memory, spot instance eligible	t3.medium, c5.large	t3.large provides 2 vCPU, 8GB RAM - sufficient for API workloads
Database - Core	RDS PostgreSQL	14+	Managed ACID	Aurora PostgreSQL	RDS is cost-effective;

			compliance, automated backups, Multi-AZ		Aurora premium not justified for asset volume
Database - Document	MongoDB Atlas	6+	Managed document DB, flexible schema, global clusters	DynamoDB, Couchbase	Atlas avoids vendor lock-in; better developer experience
Database - Time-Series	InfluxDB Cloud	2.x	Optimized for telemetry, high write throughput, downsampling	TimescaleDB, Prometheus	InfluxDB has better compression and query language for time-series
Database - Graph	Neo4j Aura	5.x	Best graph query performance, managed service	Amazon Neptune	Aura has free tier for development; Neptune would lock to AWS
Cache	ElastiCache Redis	7.x	Managed Redis, cluster mode, high performance	Memcached, self-hosted	Redis supports data structures needed for rate limiting and sessions
Message Broker	MSK (Kafka)	3.x	Managed Kafka, high throughput, durability	RabbitMQ, SQS	Kafka needed for CDC and event sourcing;

					MSK reduces ops burden
CDC	Debezium on MSK Connect	2.x	Native PostgreSQL support, Kafka integration	Custom CDC	Debezium is standard for CDC; managed connector reduces maintenance
Search	Elastic Cloud	8.x	Managed Elasticsearch, full-text search, analytics	OpenSearch	Elastic Cloud offers better developer experience and features
Workflow Engine	Temporal Cloud	Latest	Managed workflow engine, durable execution	Airflow, Camunda	Temporal designed for long-running stateful workflows (offboarding, fulfillment)
Batch Processing	MWAA (Airflow)	2.x	Managed Airflow, DAG scheduling, Python-native	Glue, Step Functions	Airflow is standard for batch ETL; MWAA reduces ops
AI Model Serving	SageMaker	-	Managed endpoints, auto-scaling, monitoring	Custom container, Lambda	SageMaker handles model versioning, A/B testing,

					and monitoring
API Gateway	Kong (self-managed on EKS)	3.x	Open source, plugin ecosystem, no vendor lock-in	AWS API Gateway	Kong provides more flexibility and avoids vendor lock-in
CDN	CloudFront	-	Global edge network, integration with AWS	Cloudflare	Native AWS integration; cost-effective
DNS	Route 53	-	AWS-native, integrates with ALB and CloudFront	Cloudflare	Native integration, simple management

---

Deliverable 4: Managed Services Selection and Justification

Format

- Document, 3-4 pages
- Filename: `TeamName_Managed_Services.pdf`

Description

This deliverable focuses specifically on managed services that reduce operational overhead, particularly for the AI/ML components and polyglot persistence layer. It

answers: Which services will be fully managed? Why is managed preferred over self-hosted? What are the cost implications?

Requirements

1. Managed Services Decision Matrix

Service Type	Self-Hosted Alternative	Managed Service	Selection	Rationale
PostgreSQL	Self-managed on EC2	RDS	✓ Managed	Automated backups, Multi-AZ failover, patch management, monitoring
MongoDB	Self-managed on EC2	MongoDB Atlas	✓ Managed	Automatic sharding, backups, global clusters, reduces DBA overhead
Redis	Self-managed on EC2	ElastiCache	✓ Managed	Automatic failover, backup/restore, monitoring, patching
Kafka	Self-managed on EC2	MSK	✓ Managed	Automated provisioning, Apache Kafka compatible, reduced ops
Airflow	Self-managed on EC2	MWAA	✓ Managed	Managed workers, automatic scaling, security patching
Kubernetes	Self-managed K8s	EKS	✓ Managed	Managed control plane, updates, security patches

InfluxDB	Self-managed on EC2	InfluxDB Cloud	✓ Managed	Serverless, automatic scaling, built-in downsampling
Neo4j	Self-managed on EC2	Neo4j Aura	✓ Managed	Automated backups, scaling, security updates
Elasticsearch	Self-managed on EC2	Elastic Cloud	✓ Managed	Automated upgrades, security, snapshot management
Temporal	Self-managed on EC2	Temporal Cloud	✓ Managed	Durable execution, scaling, monitoring

2. AI/ML Managed Services

Service	Purpose	Managed Offering	Key Features
Model Training	Demand forecasting model	SageMaker Training	Managed training instances, automatic model tuning, distributed training
Model Hosting	Real-time predictions	SageMaker Endpoints	Auto-scaling, A/B testing, model monitoring, explainability
Data Labeling	Damage detection training	SageMaker Ground Truth	Active learning, workforce management, quality control
Feature Store	ML features for forecasting	SageMaker Feature Store	Online/offline storage, feature versioning, consistency

3. Cost vs. Operational Overhead Analysis

Component	Self-Hosted Monthly Cost	Managed Monthly Cost	Operational Hours Saved (month)	Recommendation
PostgreSQL	~$150 (EC2 + storage)	~$250 (RDS)	20 hours	Managed
MongoDB	~$200 (EC2 cluster)	~$300 (Atlas)	15 hours	Managed
Kafka	~$300 (3 brokers)	~$400 (MSK)	25 hours	Managed
Kubernetes	~$200 (control plane)	~$75 (EKS)	30 hours	Managed
Total	~$850	~$1,025	90 hours	Managed (worth $9,000/month in engineer time)

Deliverable 5: Infrastructure as Code Strategy

Format

- Document with code examples (3-4 pages)
- Filename: `TeamName_IaC_Strategy.pdf`

Description

This deliverable defines the Infrastructure as Code (IaC) approach for provisioning and managing the technology architecture. It answers: How will infrastructure be defined? How will changes be reviewed and applied? How are environments managed?

Requirements

1. IaC Tool Selection

Tool	Purpose	Selection Rationale
Terraform	Primary IaC for all AWS resources	Cloud-agnostic, mature state management, large provider ecosystem, team familiarity
Terragrunt	Terraform wrapper for multiple environments	DRY configuration, remote state management, environment segregation
GitHub Actions	CI/CD for infrastructure changes	Integrated with source control, pull request workflows, cost-effective

2. Terraform Configuration Structure

```
text
terraform/
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   ├── staging/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   └── prod/
│       ├── main.tf
```

```
│       ├── variables.tf
│       └── terraform.tfvars
├── modules/
│   ├── networking/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── compute/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── database/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── security/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
└── global/
    ├── iam/
    ├── s3-backend/
    └── route53/
```

3. Terraform Configuration Example

```hcl
# modules/database/rds/main.tf
resource "aws_db_instance" "inventory_db" {
  identifier     = "inventory-db-${var.environment}"
  engine         = "postgres"
  engine_version = "14.9"
  instance_class = var.instance_class

  allocated_storage     = var.allocated_storage
  storage_encrypted     = true
  storage_type          = "gp3"

  db_name  = var.db_name
```

```
  username = var.db_username
  password = random_password.db_password.result

  vpc_security_group_ids = [aws_security_group.rds.id]
  db_subnet_group_name   = aws_db_subnet_group.main.name

  backup_retention_period = var.backup_retention_days
  backup_window           = "03:00-04:00"
  maintenance_window      = "sun:04:00-sun:05:00"

  multi_az                = var.environment == "prod"
  skip_final_snapshot     = var.environment != "prod"

  tags = {
    Name        = "inventory-db-${var.environment}"
    Environment = var.environment
    ManagedBy   = "terraform"
  }
}

# environments/prod/terraform.tfvars
environment         = "prod"
region              = "ap-southeast-1"
vpc_cidr            = "10.0.0.0/16"
db_instance_class   = "db.t3.large"
db_allocated_storage = 100
backup_retention_days = 30
```

4. Infrastructure CI/CD Pipeline

Stage	Action	Tool	Gate
Plan	`terraform plan` on PR	GitHub Actions	Review comment
Review	Manual approval for production	GitHub PR	2 approvals required

Stage	Action	Tool	Gate
Apply (Dev)	Auto-apply on merge to dev	GitHub Actions	Tests pass
Apply (Staging)	Auto-apply on tag	GitHub Actions	Dev deploy successful
Apply (Prod)	Manual trigger with approval	GitHub Actions	Staging verified

---

Deliverable 6: Scalability and High Availability Design

Format

- Document, 3-4 pages
- Filename: `TeamName_Scalability_HA.pdf`

Description

This deliverable defines how the system scales to meet demand and remains available during failures. It answers: How does each component scale? What are the SLAs? What happens during regional failure?

Requirements

1. Component Scalability Matrix

Component	Scaling Type	Scaling Trigger	Min Instances	Max Instances	Scale-up Time
API Application	Horizontal (HPA)	CPU > 70%, Memory > 80%	3	20	2-3 minutes
Predictive Service	Horizontal (HPA)	CPU > 60%, Queue length > 100	2	10	2-3 minutes
Telemetry Service	Horizontal (HPA)	CPU > 70%, Messages/sec > 1000	2	15	2-3 minutes
PostgreSQL	Vertical	Read replica count, storage	1 primary + 1 replica	1 + 5 replicas	10-15 minutes
MongoDB	Horizontal (sharding)	Storage > 80%, Write throughput	3 nodes	Unlimited	5-10 minutes
Kafka	Horizontal (partitions)	Partition lag > 10000	3 brokers	12 brokers	5 minutes
Redis	Cluster (sharding)	Memory > 75%	1 primary + 2 replicas	15 shards	5 minutes

2. High Availability Configuration

Component	HA Strategy	RTO (Recovery Time)	RPO (Recovery Point)	SLA
API Application	Multi-AZ deployment, load balancer	< 1 minute	0 (stateless)	99.95%
PostgreSQL	Multi-AZ with automatic failover	< 2 minutes	< 5 seconds (sync replica)	99.95%
MongoDB	Multi-zone cluster (3 nodes)	< 1 minute	0 (majority write)	99.99%
Kafka	Multi-AZ, replication factor 3	< 2 minutes	0 (acks=all)	99.9%
Redis	Cluster with replicas in each AZ	< 1 minute	< 1 second	99.9%
InfluxDB	Cloud-managed multi-zone	< 2 minutes	< 1 minute	99.95%
Neo4j	Cloud-managed multi-zone	< 2 minutes	< 5 minutes	99.95%

3. Disaster Recovery Plan

Tier	Recovery Objective	Strategy	Annual Testing
Critical (Order processing, inventory)	RTO < 1 hour, RPO < 5 minutes	Cross-region replication, active-passive	Quarterly

Tier	Recovery Objective	Strategy	Annual Testing
Important (User data, assignments)	RTO < 4 hours, RPO < 1 hour	Daily backups to separate region, point-in-time recovery	Semi-annual
Normal (Logs, telemetry)	RTO < 24 hours, RPO < 24 hours	Weekly backups	Annual

---

Deliverable 7: Cost Estimation and Budget Analysis

Format

- Spreadsheet
- Filename: `TeamName_Cost_Estimate.xlsx`

Description

This deliverable provides a detailed cost estimate for the technology architecture, including monthly and annual projections. It answers: How much will this cost? What are the major cost drivers? How can costs be optimized?

Requirements

1. Monthly Cost Breakdown

Service	Instance Type	Quantity	Unit Price (USD)	Monthly Total (USD)	Annual Total (USD)
Compute					
EKS Cluster	Control plane	1	$72	$72	$864
EKS Worker Nodes (API)	t3.large (spot)	3	$35	$105	$1,260
EKS Worker Nodes (Predictive)	t3.medium (spot)	2	$17.50	$35	$420
EKS Worker Nodes (Telemetry)	t3.medium (spot)	2	$17.50	$35	$420
Databases					
RDS PostgreSQL	db.t3.large (Multi-AZ)	1	$150	$150	$1,800
RDS Storage	gp3, 100GB	1	$10	$10	$120
MongoDB Atlas	M30 (3 nodes)	1	$300	$300	$3,600
InfluxDB Cloud	Starter tier	1	$49	$49	$588

Service	Instance Type	Quantity	Unit Price (USD)	Monthly Total (USD)	Annual Total (USD)
Neo4j Aura	Professional	1	$199	$199	$2,388
ElastiCache Redis	cache.t3.micro (cluster)	3	$12	$36	$432
Integration					
MSK Kafka	kafka.t3.small (3 brokers)	1	$150	$150	$1,800
MSK Storage	100GB per broker	3	$10	$30	$360
MWAA Airflow	mwaa.small	1	$150	$150	$1,800
AI/ML					
SageMaker Endpoint	ml.t3.medium	1	$90	$90	$1,080
SageMaker Training	ml.t3.large (occasional)	-	$20	$20	$240
Networking					
NAT Gateway	-	2	$32	$64	$768
Data Transfer	1 TB	-	$90	$90	$1,080
CloudFront	1 TB	-	$85	$85	$1,020
Load Balancer	ALB	1	$22	$22	$264

Service	Instance Type	Quantity	Unit Price (USD)	Monthly Total (USD)	Annual Total (USD)
Storage & Backup					
S3 (static assets)	10 GB	1	$0.23	$0.23	$2.76
RDS Backup Storage	100 GB	1	$10	$10	$120
Management & Security					
AWS WAF	-	1	$25	$25	$300
AWS Shield Advanced	-	1	$3,000	$3,000	$36,000
TOTAL				**$4,927.23**	**$59,126.76**

2. Cost Optimization Strategies

Strategy	Description	Estimated Savings	Priority
Spot Instances	Use spot instances for non-critical workloads (batch, dev)	60-70% on compute	High

Reserved Instances	Purchase 1-year RI for RDS and consistent workloads	30-40% on databases	Medium	
Auto-scaling	Scale down to minimum during off-hours	40% on compute	High	
Data Lifecycle	Move old telemetry to cold storage after 30 days	50% on storage	Medium	
Graviton Processors	Use AWS Graviton (ARM) instances where compatible	20% on compute	Low	
Savings Plan	AWS Compute Savings Plan for predictable usage	25% across compute	Medium	
Optimized Total			**~$3,200/month**	**~$38,400/year**

Deliverable 8: Traceability Matrix (Technology Decisions to Requirements)

Format

- Spreadsheet
- Filename: `TeamName_Tech_Traceability.xlsx`

Description

The Traceability Matrix links technology architecture decisions back to the pain points from Activity 1, TO-BE processes from Activity 2, and application architecture from Activity 4. This ensures that every technology choice is justified by business need.

Requirements

1. Pain Point to Technology Solution Traceability

Pain Point ID	Pain Point Description	Technology Solution	Component	Justification
P1	Procurement-to-inventory delay	RDS + MSK + Debezium	Data Layer	CDC enables real-time asset creation from procurement events
P3	Poor mobile experience	CloudFront + S3	Delivery	CDN ensures fast mobile app asset delivery globally
P5	License over-purchasing	RDS + Elasticsearch	Search/Analytics	Fast search enables license usage tracking and compliance reporting
P7	Asset non-return after offboarding	Temporal Cloud + RDS	Workflow	Durable workflow ensures offboarding steps complete with retries

Pain Point ID	Pain Point Description	Technology Solution	Component	Justification
P9	Finance spreadsheets	RDS + MWAA	Batch	Automated monthly exports to ERP via Airflow DAGs
P12	Audit findings	RDS + Neo4j + Elasticsearch	All	Complete audit trail and relationship mapping for auditors
P15	Predictive failures	InfluxDB + SageMaker	AI/Telemetry	Time-series database enables ML-based failure prediction

2. TO-BE Process to Technology Support Traceability

TO-BE Process	Technology Support	Components	Deployment
1.0 User Sync	HRIS webhook → API → RDS	API on EKS, RDS	Private subnet
2.0 Procurement Intake	Procurement API → MSK → RDS	MSK, Debezium, RDS	Private + Data subnets
3.0 Telemetry Ingestion	MDM → Telemetry Service → InfluxDB	EKS, InfluxDB Cloud	Private subnet + SaaS
4.0 AI Analytics	InfluxDB → SageMaker → Kafka	SageMaker, MSK, EKS	AI/ML subnet
5.0 Network Mapping	Network Scanner → Neo4j	Neo4j Aura, EKS	Private subnet + SaaS

TO-BE Process	Technology Support	Components	Deployment
6.0 License Management	RDS → Elasticsearch	Elastic Cloud, RDS	Private subnet + SaaS
7.0 Asset Retirement	API → Temporal → RDS	Temporal Cloud, EKS, RDS	Private subnet + SaaS
